

友元函数

1. 类外部不能访问该类的 private 成员
 1. 通过该类的 public 方法
 2. 会降低对 private 成员的访问效率，缺乏灵活性:如果使用public方法使用这些成员则是实行调用函数，降低调用效率，消耗时间
 3. 例：矩阵类(Matrix)、向量类(Vector)和全局函数(multiply)，全局函数实现矩阵和向量相乘
 4. 隐藏细节、保持一致性
 2. 友元是数据保护和访问效率的折衷方案
 3. 友元可以访问private和protected的成员
- [1. Example:Matrix 和 Vector](#)
 - [2. 友元的分类](#)
 - [2.1. 友元函数](#)
 - [2.2. 友元类](#)
 - [2.3. 友元类成员函数](#)
 - [3. 友元函数分类实例](#)
 - [4. 友元函数声明](#)
 - [5. 友元函数作用](#)
 - [6. 声明两个类互为友元](#)
 - [7. 封装原则](#)
 - [8. 友元和protected](#)

1. Example:Matrix 和 Vector

```
1 //Matrix
2 class Matrix{
3     int *p_data;//逻辑二维，一维存储
4     int lin,col;
5     public:
6         Matrix(int l, int c){
7             lin = l;
8             col = c;
9             p_data = new int[lin*col];
10        }
11        ~Matrix(){
12            delete []p_data;
13        }
14        int &element(int i, int j){
15            return *(p_data+i*col+j);//指针类型的偏移是根据指针指向对象的类型
16        }
17        void dimension(int &l, int &c){
18            l = lin;
19            c = col;
20        }
21        void display(){
```

```

22         int *p=p_data;
23         for (int i=0; i<lin; i++){
24             for (int j=0; j<col; j++){
25                 cout << *p << ' ';
26                 p++;
27             }
28             cout << endl;
29         }
30     };
31 };
32 //Vector
33 class Vector{
34     int *p_data;
35     int num;
36     public:
37     vector(int n){
38         num = n;
39         p_data = new int[num];
40     }
41     ~Vector(){
42         delete []p_data;
43     }
44     int &element(int i)
45     { return p_data[i]; }
46     void dimension(int &n)
47     { n = num; }
48     void display(){
49         int *p=p_data;
50         for (int i=0; i<num; i++,p++)
51             cout << *p << ' ';
52         cout << endl;
53     }
54 };
55 //实现 矩阵和一个向量进行计算，效率比较
56 void multiply(Matrix &m, Vector &v, Vector &r){
57     int lin, col;
58     m.dimension(lin,col);
59     for (int i=0; i<lin; i++){
60         r.element(i) = 0;
61         for (int j=0; j<col; j++)
62             r.element(i) += m.element(i,j)*v.element(j); //这里的调用效率会比较低
63     }
64 }
65
66 void main(){
67     Matrix m(10,5);
68     Vector v(5);
69     Vector r(10);
70     multiply(m,v,r);
71     m.display();
72     v.display();
73     r.display();
74 }

```

2. 友元的分类

2.1. 友元函数

1. 一个全局函数是一个类的友元，如果在这之前没有声明也是可以进行声明友元函数。

2.2. 友元类

1. 一个类是另一个类的友元
2. class关键字可以省略
3. 第一种情况:friend class B:
 1. 编译器会寻找有没有类B
 2. 如果没有则会引入一个B
4. 第二种情况:friend B
 1. 省略关键字的时候不会引入B，如果没有B会报错模板类
 2. 但是这种形式常用于模板类(T或者typedef的时候来写)

2.3. 友元类成员函数

1. 在完整的类的声明完成之前是不能够被声明的。

3. 友元函数分类实例

```
1 void func() ;
2 class B; //这种情况下B不是必须的
3 class C{
4     void f();
5 };
6 class A{
7     friend void func(); //友元函数
8     friend class B;      //友元类:B中的每一个函数都可以访问A的成员函数
9     friend void C::f(); //友元类成员函数
10};
```

4. 友元函数声明

1. 友元函数在之前可以没有声明
2. 友元函数如果之前还没有声明过，则当做已经声明了
3. 但是友元类函数在完整的类声明出现前不能声明友元函数。
4. 为什么友元函数和友元类成员函数的声明要求是不一样的?
 1. 数据的一致性:避免对应类里面没有这个函数(也就是C的完整定义必须有)
 2. 成员函数依赖于类

5. 友元函数作用

1. 作用

1. 提高程序设计灵活性
2. 数据保护和对数据的存取效率之间的一个折中方案

2. 友元不具有传递性:

1. 不能说A是B的友元, B是C的友元就可以得出A是C的友元
2. 友元必须显式声明

```
1 class Matrix{
2     friend void multiply(Matrix &m, Vector &v, Vector &r);
3 };
4 class Vector{
5     friend void multiply(Matrix &m, Vector &v, Vector &r);
6 };
```

1. 上面这段代码可以编译吗? (循环依赖)

1. 不可以编译的, 要在前面先声明Vector
2. 使用变量前必须要先声明
3. Matrix里面如果去掉Vector中的引用? 会出现内存分配问题(不知道如何拷贝, 而引用大小是相同的)

2. 解决方案:不完全声明: `class vector;`

6. 声明两个类互为友元

```
1 class A{
2     int a;
3     public:
4         friend class B;
5     void show(B &b){
6         std::cout << b.b; //这里可以吗? 不行, 不知道B中有b
7     }
8 }
9 class B{
10     int b;
11     public:
12         friend class A;
13     void show(A &a){
14         std::cout << a.a; //这里是可行的
15     }
16 }
17 void A::show (B &b){ //只能在这里面实现
18     std::cout << b.b;
19 }
```

1. 互为友元的两个类声明时是否需要前置声明

1. 如果A和B不在一个命名空间不能通过编译
2. 如果A和B在一个命名空间的话可以没有前置声明

7. 封装原则

1. 避免将data member放在公开接口中(使用get和set方法)

1. 尽量将get和set成员更加完备化

2. 遵循迪米特法则(最小知识原则):尽量让别的类对当前类的依赖最小

```
1 class AccessLevels {
2     public:
3         int getReadOnly const { return readOnly; }
4         void setReadWrite(int value) { readwrite = value; }
5         int getReadWrite() { return readwrite; }
6         void setWriteOnly(int value) { writeonly = value; }
7     private:
8         int noAccess;
9         int readOnly;
10        int readwrite;
11        int writeonly;
12};
```

2. 努力让接口完满 (complete) 且最小化

8. 友元和protected

```
1 class Base{
2     protected :
3         int prot_mem; // protected 成员
4 };
5 class Sneaky : public Base { //36min
6     friend void clobber(Sneaky&); //能访问Sneaky::prot_mem
7     friend void clobber(Base&); //不能访问Base::prot_mem, 对外不可见
8     int j; // j 默认是private
9
10    void clobber(Sneaky &s) {
11        s.j = s.prot_mem = 0;
12    } //正确: clobber 能访问Sneaky对象的 private和protected成员
13    void clobber(Base &b) {
14        b.prot_mem = 0;
15    } //错误:clobber 不能访问Base的 protected 成员
16 }
```

1. 继承过程中的友元传递:友元不具有传递性, 不可以访问任意基类的